Java Script Test:

Sarjith. S
F.E. - 16th batch
4.10.2026
EID : 7388

1. Variable :

* It is a storge used to store data in Programs.

* By declaring avariable, we can use it or anywhere on the Program with its variable name for easy access.

* It has 3 declaration types.

Var - oldest method that uses both reassign and redeclaration, results in conflict.

Let - Best method that uses reassign but not redeclaration, Prevents us from using same name for different datas.

Const - It is used when you don't want its value to be changed throughout the Program.

2. Java Script data types :

Values that can be Stored inside the variables are classified into five types.

String:

Collection of characters.

Ex: let StringOne = "JavaScript"

Number:

Numerical values, mostly only Natural values.

Ex: let newNumber = 100

Boolean:

Simple 0s or 1s. (True or false)

Ex: Let BooleanValue = true

Note: If you use double quoter for the 'true', it will become a string.

Undefined:

Empty variable where no data is given

Ex: let nameString;

the log output will be 'undefined'

Null:

Intentionally declared null by the developer.

Ex: let UserName = null

3) difference b/w == and ===

== → It checks whether the two values are same regardless of its data type.

Ex: let a = 10,
let b = '10'
a == b

the output will be true since the value is 10.

=== → It checks both the datatype and the value.

Ex: let a = 10
let b = '10'
a === b

the output will be false because a is Number and b is a string.

4) Operators:

Arithmetic operators:

These operators are used to Perform mathematical operations, ex: +, -, *, /, %, **.

Ex: c = 10 + 10 ⇒ 20
c = 10 - 10 ⇒ 0
c = 10 * 10 ⇒ 100
c = 10 / 10 ⇒ 1
c = 10 % 10 ⇒ 0
c = 10 ** 10 = 100

Assignment operators:

Core operator for assigning values to the variable. It is used to store your required value on a variable.

Ex: =, +=, -=, *=, /=, %=, **=

```
let a = 20
let b = 10
log (a = b  => a = 10,
a += b  => a = a+b  => a = 30
a -= b  => a = a-b  => a = 10
a *= b  => a = 200
a /= b  => a = 2
```

Comparision operation:

mainly used on Conditional and Control statements. It checks a condition and return either true or false. Ex: ==, ===, <, >, <=, >=, !=,

```
!==
let a = 20
let b = 10
log (a == b)  => false
a <= b  => false
a >= b  => true
```

logical operator:

used to add two or more condition at the same time. Ex: &&, ||, !.

```
let a = 20
let b = 10
a != b && a <= b  => false.
a != b || a <= b  => true.
a !== b  !(a==b) => true.
```

&& - both the condition should be true.
|| - atleast one condition is true
! - reverser the condition result.

5. DIB:

Array:

Multiple Number of different types of data can be stored inside a single variable. With the help of square brackets "[]" and the datas are separated by ~~sp~~comma ",".
* They are iteratable using its index value

```
let array = [ 1, 2, "react", 4, true ]
              0  1   2      3   4  → index values.
```

Objects:

It also stores multiple datas but instead of navigating them through index value, we we the key for identification. It is enclosed by "{}"
* they are not Iteratable.

```
let object = {name : "john", age: 17, skills : "JS"}
```

Array of objects:

Objects inside the array to store number of objects or arrays.

```
let arrayObj = [ {name : "john", age : 17}, {name: "chris", age=14} ]
```

* we can also use array inside the object.

6. Function:

Function is a manmade shortcut that is used to create reusable codes. Once a function is created, we can use the function name to call that function from any where inside the Program.

Ex:
```
let a = 20
let b = 10
const additionProb = (c, d) => {
    let tot = c+d
    return tot
}
console.log (additionProb (a,b));
```
output : 30.

Parameter:

the values that we give in the code while invoking a functions are called as Parameters. It either accepts values or are variables.

log (function (a,b)) ; → Parameters.

Arguments:

the variable name declared inside the function definition are called or arguments.

const addition=(c,d) => { } → arguments.

Return:

* Kill switch for a function.
* It always returns a value that will be stored in memory
* any line or command inside function below the return will not work.

PART - B:

7.

Even or odd :

```
let a = 27
if ( a % 2 ==0) {
        log ("true");
else {
      log("false")
}
```

8. loop - for 1 to 50 :

```
for (let a = 1 ; a <=50; a++) {
      log (a)
}
```

in single line :

```
let line = " "
for (let a = 1 ; a <=50; a++) {
      line += a + " "
}
log ( line )
```

9) Factorial :

```
let sol = 1
let number = 5
for ( let a = 1 ; a <= number ; a++) {
      sol *= a
}
log( sol)
```

10. Fibonacci Series:

```
let one = 0
let two = 1
let three = 0
let line = one + "," + two + ","
for (let a=0 ; a <10; a++) {
    three = one + two
    line += three + ","
    one = two
    two = three
}
log (line)
```

Note: If you want to remove the "," at the end,

replace: `line += three + ","`

with:

```
if( a == 9) {
    line += three
}
else {
    line += three + ","
}
```

PART-C:

11)

```
array:
let even = []
let numbers = [10, 15, 20, 25, 30, 35, 40]
let count = 0
for (let i=0; i < numbers.length; i++) {
    if (numbers[i] % 2 == 0) {
        even[count] = numbers[i]
        count ++
    }
}
log (even)
```

12) Array total:

```
let total = 0
let array = [80, 75, 90, 85, 70]
for (let i=0; i < array.length; i++) {
    total += array[i]
}
log (total).
```

13) object:

```
let object = { name: "john", age: 17, Course: "fullstack",
               mark: 75 }

log (object.name)
log (object.Course)
log (object.mark)
```

14. Array of object:

```
let students = [ {name : "arun", mark : 80} ,
                 {name : "kumar", mark : 45},
                 {name : "Priya", mark : 90},
                 {name : "Ram", mark : 35} ]
let result = []
let c=0
for (let a = 0; a < students.length ; a++){
      if (students[a].mark >=50) {
          result [c] = student [a]
          c ++
      }
}
log (result);
```

7388

PART-D:

05. Largest number:

```
let a = 20
let b = 10

Const greatestNumber = (a, b) => {
      if (a > b) {
          return a
      }
      else {
          return b
      }
}
log ( greatestNumber(a, b) )
log ( greatestNumber (5, 10) )
```